Name:	**Jay Talekar**
Roll No:	**50**
Class/Sem:	TE/V
Experiment No.:	05
Title:	To Containerize applications using Docker
Date of Performance:	
Date of Submission:	

Marks:

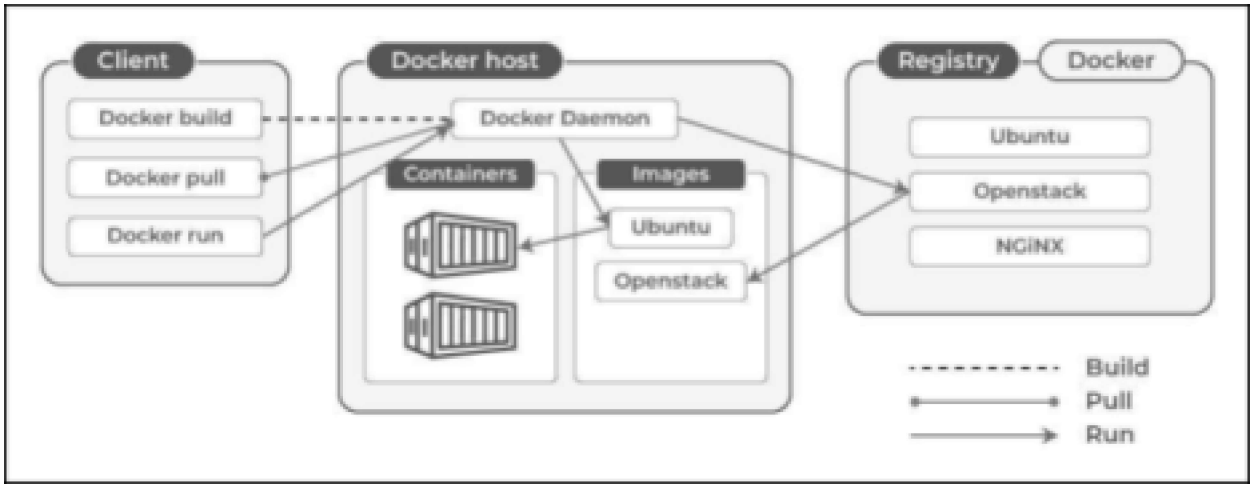
Sign of Faculty:

Aim: To Containerize applications using Docker

Objective: To understand Docker architecture and the lifecycle of containers and to learn essential Docker commands for managing images and interacting with containers. This enables users to package, distribute, and run applications consistently across different environments.

Theory: Docker is a containerization technology that allows developers to package their applications and dependencies into lightweight, portable containers. These containers can then be run on any system that supports Docker, making it easier to deploy and manage applications in various environments.

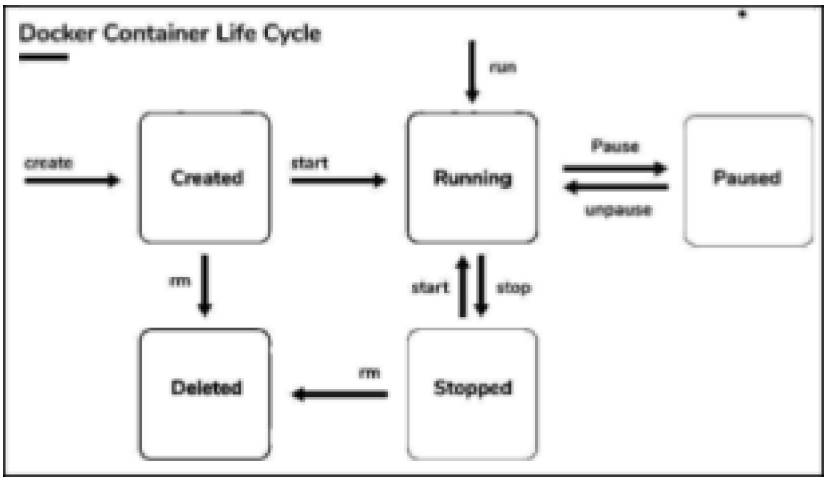
Docker Architecture:



The Docker architecture consists of the following components:

1. **Docker Client:** This is the primary interface that developers use to interact with Docker. It sends commands to the Docker daemon and displays the output.
2. **Docker Daemon:** This is the background process that runs on the host machine and manages Docker objects such as images, containers, networks, and volumes.
3. **Docker Registry:** This is a storage and distribution system for Docker images. It allows developers to share and distribute their images with others.
4. **Docker Image:** This is a lightweight, standalone, executable package that includes everything needed to run an application, including the application code, runtime, libraries, dependencies, and system tools.
5. **Docker Container:** This is a runtime instance of a Docker image. It is isolated from the host system and other containers, providing a secure and predictable environment for the application to run in

Container Lifecycle:

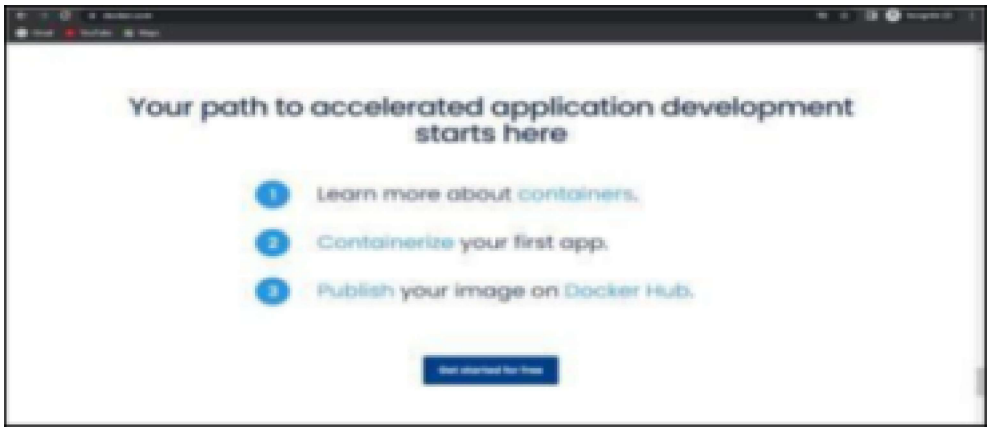


The lifecycle of a Docker container consists of the following stages:

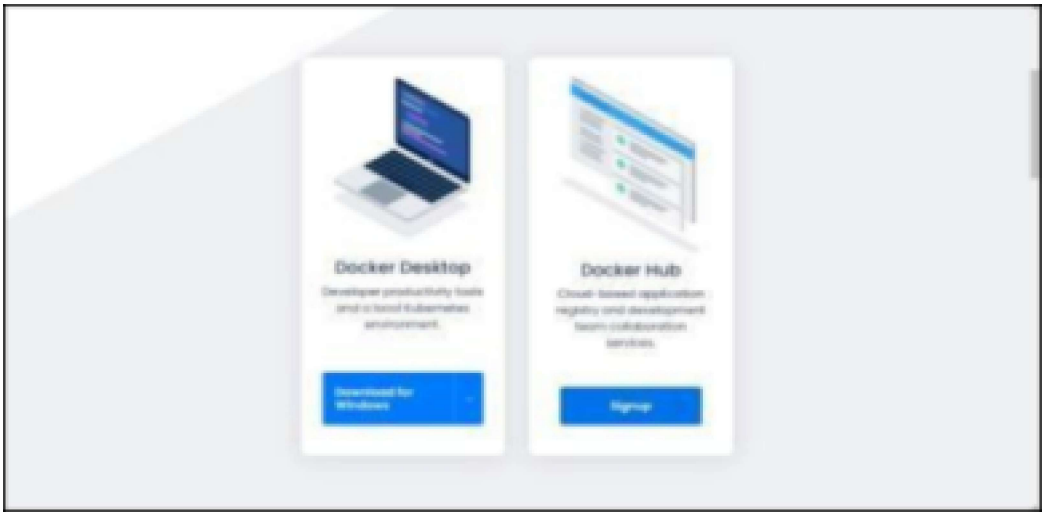
1. **Create:** To create a container, you start by creating an image that includes all the necessary components to run the application. This image is then used to create the container.
2. **Start:** Once the container is created, you can start it using the docker start command. This launches the container and runs the application inside it.
3. **Run:** Once the container is started, you can interact with the application inside the container. You can run commands, access files, and make changes to the application.
4. **Stop:** When you're done using the container, you can stop it using the docker stop command. This stops the application inside the container and shuts down the container.
5. **Remove:** Finally, when you no longer need the container, you can remove it using the docker rm command. This removes the container from the system and frees up any resources it was using.

Steps for Installation:

Step 01: Open docker.com
Scroll down, Click on 'Get started for free' tab.



Step 02: Click on Docker Desktop, Download it.



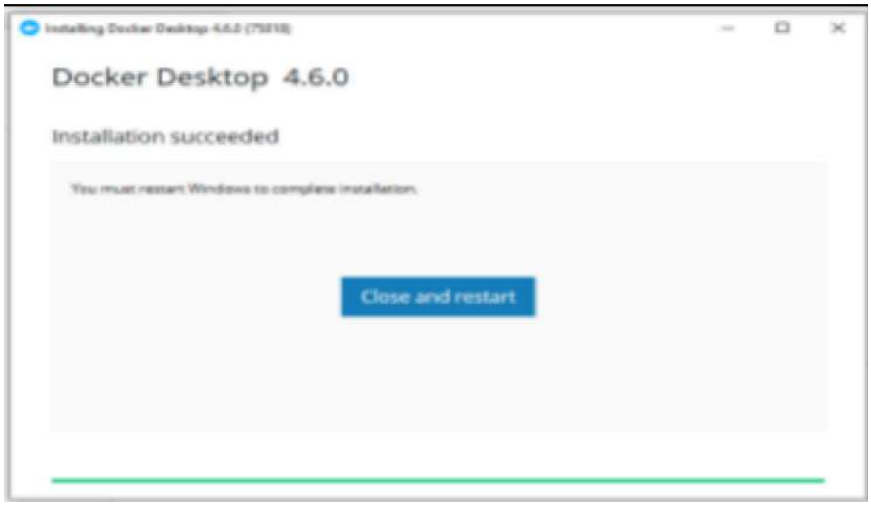
Step 03: After downloading, Open ‘Docker Deskt



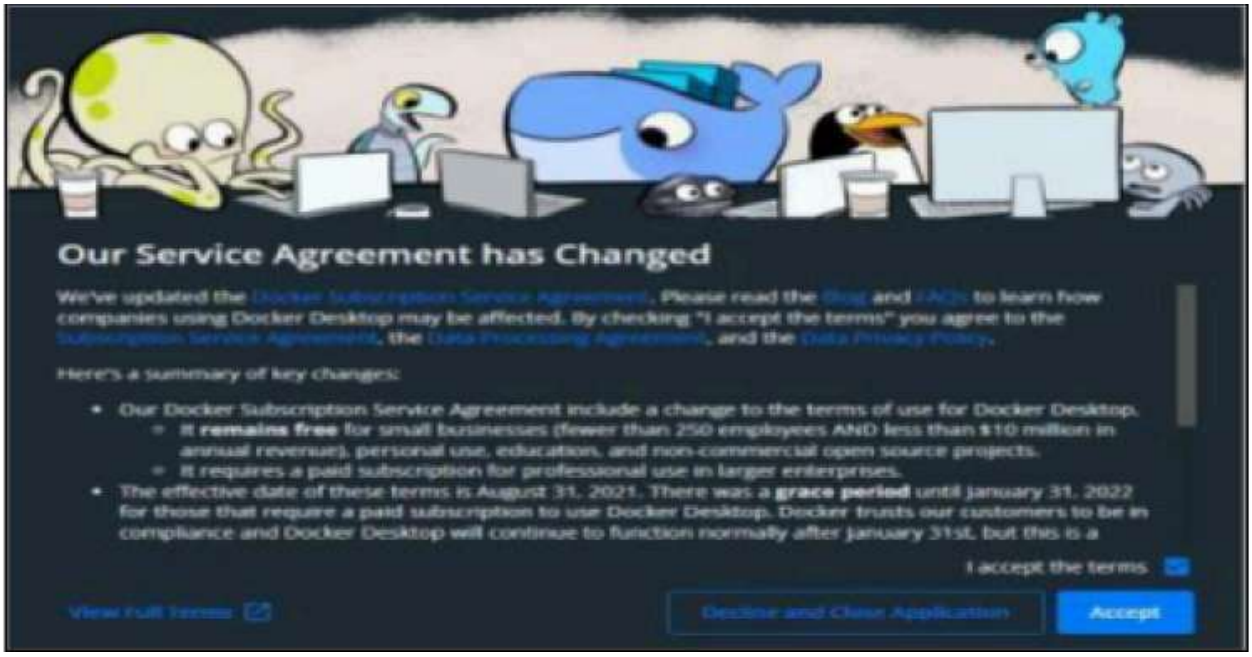


op Installer' & start installation.

Step 04: After Installation, Restart your device.



Step 05: Accept the terms and conditions, Click on Accept.



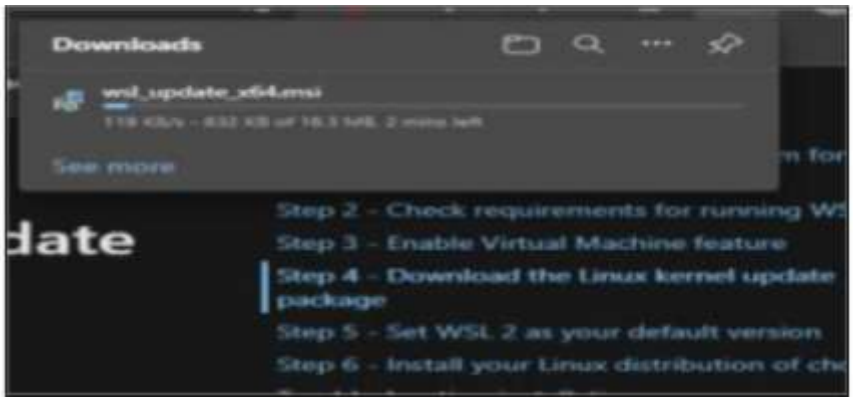


The following window should pop up.
Click on the link - https://aka.ms/wsl2kernel.
(Do not close this window).

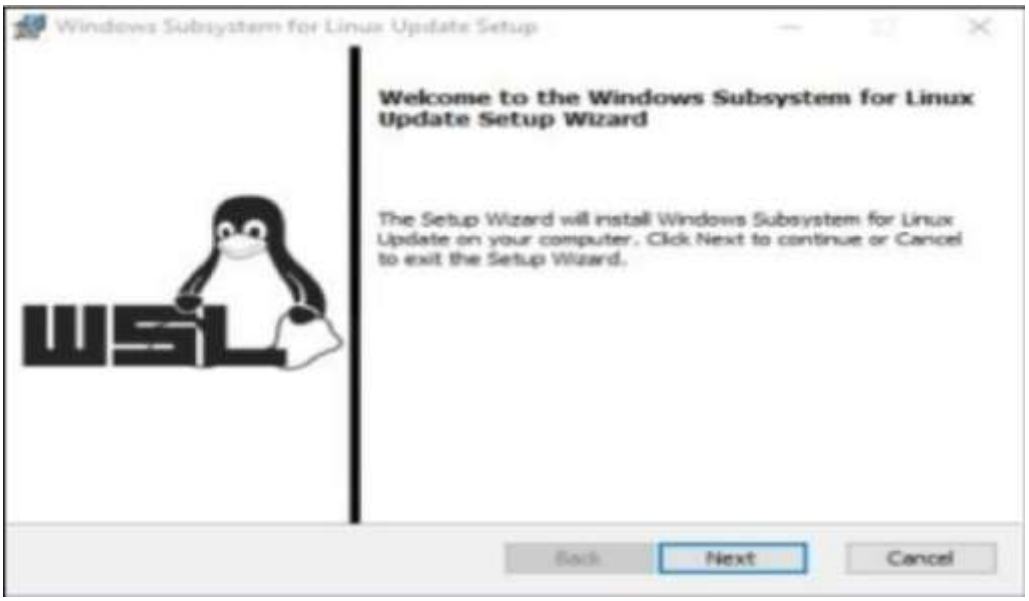


Download the WSL2 Linux kernel update package for x64 machines.

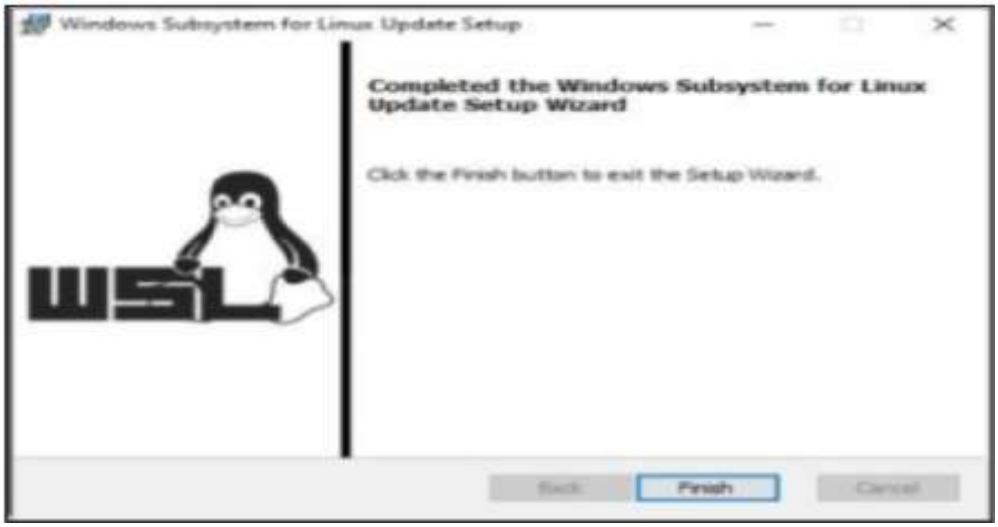




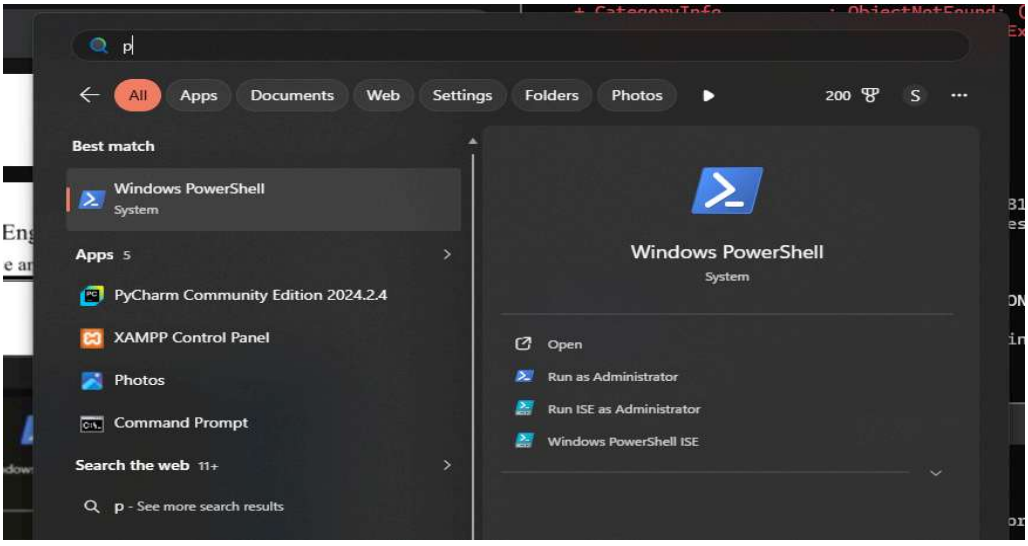
After Download is complete, Run the .msipackage.
Click on next.



After, the setup is complete, Click on finish.



Open Powershell as an Administrator.



Run the following Command:
wsl --set-default-version 2

```
PS C:\Users\student> wsl --set-default-version 2
For information on key differences with WSL 2 please visit https://aka.ms/wsl2
The operation completed successfully.
```

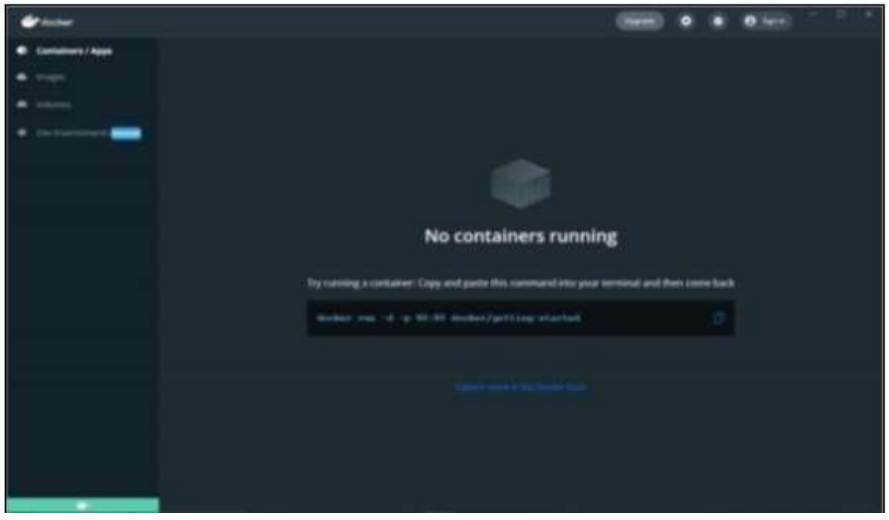
Now, Click on Restart



Docker should now restart.
Click on Start.



The following window should pop up.
This means, Installation is now complete.



Output:

Open Command Prompt, run the following commands:

1) To check the version of

Docker: docker --version

2) To install image of ubuntu docker pull ubuntu

```
C:\Users\student>docker --version
Docker version 29.5.3, build d1c06ef

C:\Users\student>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest

What's next:
    View a summary of image vulnerabilities and recommendations → docker scout quickview ubuntu
```

3) Check downloaded

images, docker images

```
C:\Users\student>docker images
                                                                    i Info →  U In Use
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest      6caf772f5178       14.1MB             0B
ubuntu:latest                        de7345b16e94        100MB             0B    U
```

4) Run ubuntu OS

docker run -it ubuntu /bin/bash

```
C:\Users\student>docker run -it ubuntu /bin/bash
root@501d3b0a5631:/#
```

5) Open another Command Prompt and follow the steps shown below.

-docker ps

docker container ls –a

docker container rm b71e3e6b1118 //copy docker id for remove but first (Use your container ID in the above command)

stop your docker

- docker container stop b71e3e6b1118
- docker container rm b71e3e6b1118
- docker ps
- docker //list all docker commands
- docker images
- docker image rm ff0fea8310f3 // copy image id from previous

output (Use your image ID in the above command)

- docker run -it ubuntu /bin/bash //check output

```
C:\Users\student>docker images
                                                                    i Info →   U  In Use
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest      6caf772f5178       14.1MB            0B
ubuntu:latest                        de7345b16e94        100MB            0B    U

C:\Users\student>docker image rm de7345b16e94
Error response from daemon: conflict: unable to delete de7345b16e94 (must be forced) - image is being used by stopped container db48ee689171

C:\Users\student>docker container ls -a
CONTAINER ID   IMAGE     COMMAND       CREATED         STATUS                     PORTS     NAMES
db48ee689171   ubuntu    "/bin/bash"   3 minutes ago   Exited (0) 3 minutes ago             priceless_brattain

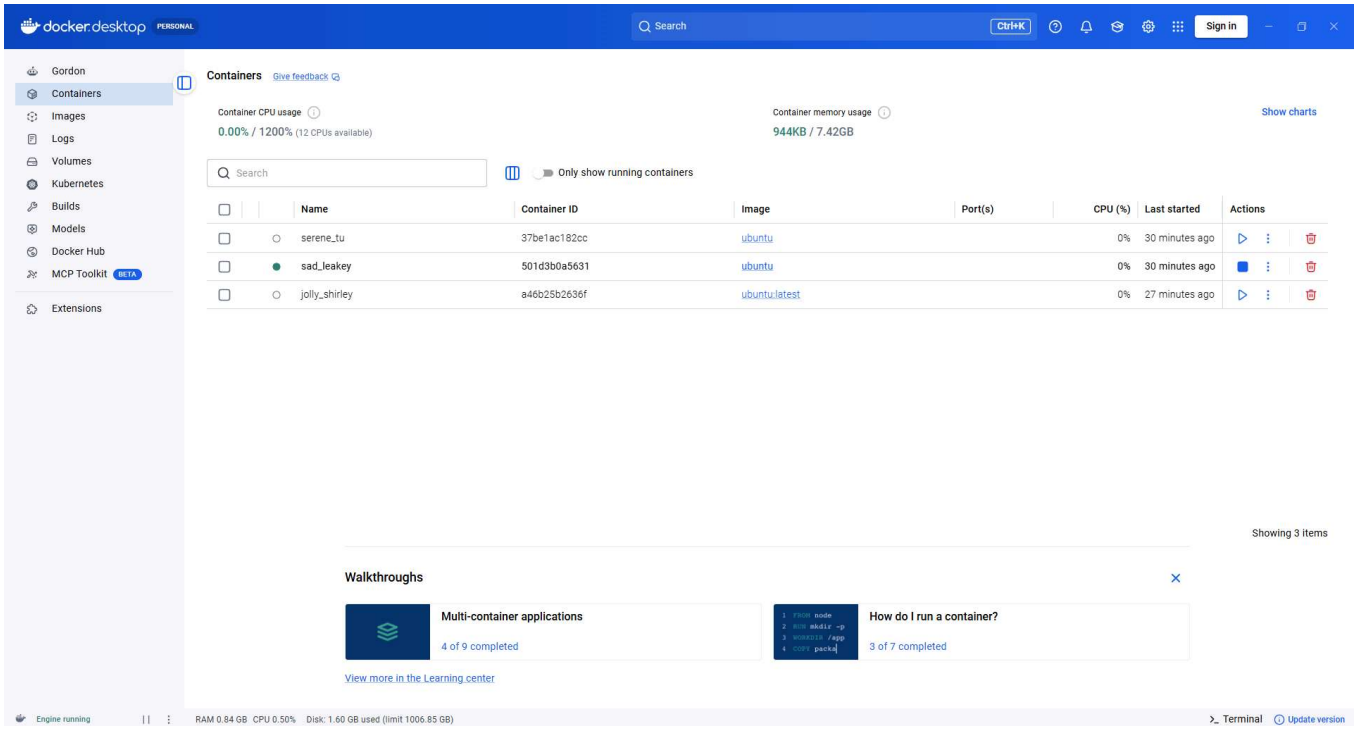
C:\Users\student>docker container rm db48ee689171
db48ee689171

C:\Users\student>docker container ls -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\student>docker image rm de7345b16e94
Untagged: ubuntu:latest
Untagged: ubuntu@sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
Deleted: sha256:de7345b16e942e22044c6ba053020ec85ae879984860a9918517d54eb6cef851
Deleted: sha256:9384d7fbfc7805e1cf888304178b7ecf01f8f1e766e1798cac368238c41b3df1
Deleted: sha256:d6b1a90bccf18353db11e206211d1438050e64d14c56a48ae067131e0a3cc245

C:\Users\student>docker run -it ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
ed819469700f: Pull complete
a3679419df18: Pull complete
Digest: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
Status: Downloaded newer image for ubuntu:latest
root@37be1ac182cc:/# exit
exit

C:\Users\student>docker images
                                                                    i Info →   U  In Use
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest      6caf772f5178       14.1MB            0B
ubuntu:latest                        de7345b16e94        100MB            0B    U
```



Conclusion:

Q1. What is the difference between containerization and virtualization?

Sol:

The core difference comes down to **what gets shared**.

- **Virtualization** virtualizes the **hardware**. Each Virtual Machine (VM) includes a full copy of a host operating system (OS), the application, and its dependencies. Because every VM runs its own separate OS kernel on top of a hypervisor, VMs are heavy, take minutes to boot, and consume significant RAM and disk space.
- **Containerization** virtualizes the **operating system**. Containers run on top of the host OS and **share the host system's Linux/Windows kernel**. They isolate the application and its dependencies in user space. Because they don't package a full OS, containers are lightweight, boot in seconds, and use far fewer system resources.

Isolation Level	Hardware-level	OS-level
Feature	**Virtualization (VMs)**	**Containerization (Docker)**
Guest OS	Full OS per virtual machine	Shares host kernel
Startup Time	Minutes	Seconds
Resource Overhead	High (GBs of memory)	Low (MBs of memory)
Best Used For	Running multiple different OS types on one server	Cloud-native apps, microservices, consistent dev environments

Q2. What is Docker Daemon?

Sol:

The **Docker Daemon** (`dockerd`) is the background service running on your host system that acts as the "brain" or engine of Docker.

When you type a command into your terminal (like `docker run` or `docker build`), you are using the **Docker CLI**. The CLI doesn't execute the work itself; instead, it sends your command as an API request to the **Docker Daemon**.

The daemon listens for API requests and manages all the heavy lifting:

- **Images:** Downloading, building, and storing Docker images.
- **Containers:** Starting, stopping, and monitoring container lifecycles.
- **Networks & Storage:** Configuring container networking, port mapping, and persistent volumes.

Because the CLI and Daemon communicate over an API, they can run on the exact same machine, or your CLI can control a remote Docker Daemon running on a cloud server.